

DES Encryption & Decryption Implementation in Python

Implementation of the Data Encryption Standard (DES) Algorithm

CRYPTOGRAPHY

PYTHON

SECURITY

Group Members

Ashenafi Bancha UGR/1796/15

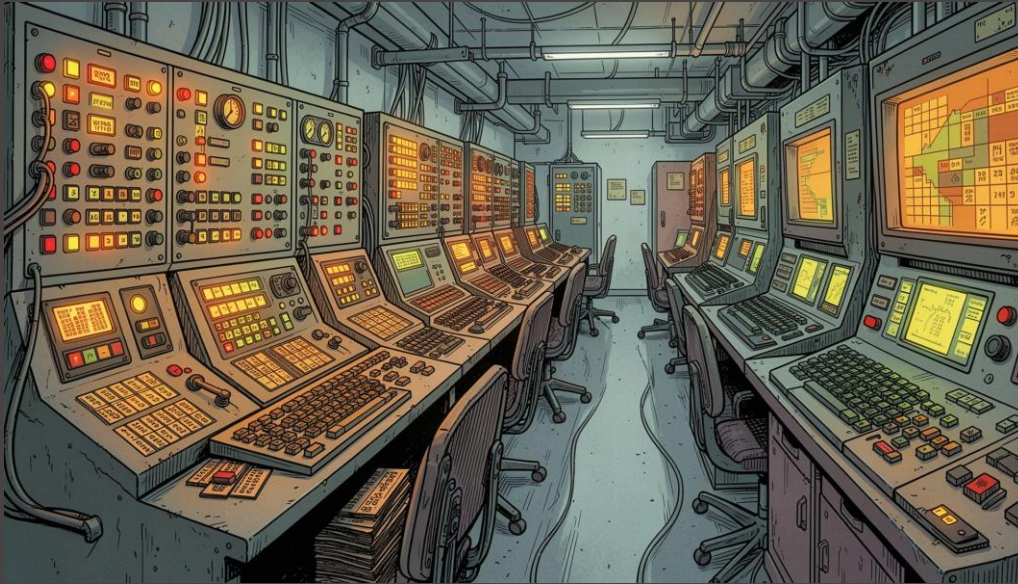
Elham Jemal UGR/1757/14

Feruza Hassen UGR/6423/15

Ihsan Jemal UGR/9433/15



Introduction: What is DES?



Overview

DES stands for **Data Encryption Standard**.

Developed by IBM in the 1970s, it is a symmetric key block cipher that became one of the most widely adopted encryption standards in history.

Key Specifications

- 64-bit block size
- 56-bit effective key length
- 16 Feistel rounds

❑ DES was once widely used for secure communication but is now considered insecure for modern systems due to its short key length.

Objectives & Features

Project Objectives

- Implement DES from scratch using Python
- Understand Feistel structure, permutations, S-box substitution, and key scheduling
- Perform encryption and decryption
- Create a command-line encryption tool

System Features

Encrypt

Plaintext converted into hexadecimal ciphertext

Decrypt

Ciphertext restored back into plaintext

16-Round Feistel

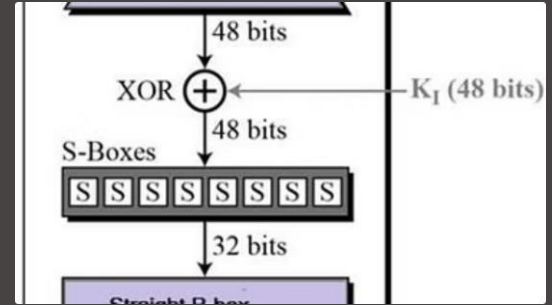
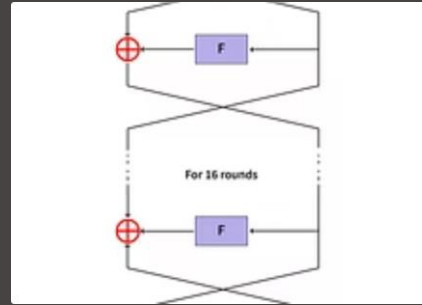
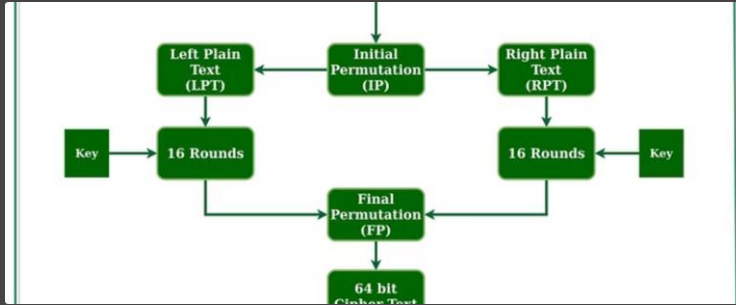
Full Feistel network with round key generation

Validation

Input validation and automated testing

DES Architecture & Encryption Flow

The DES algorithm follows a well-defined multi-step process. Plaintext is first converted to binary, then an initial permutation is applied before splitting into left and right halves. The data passes through 16 Feistel rounds, and a final permutation produces the ciphertext.



01

Binary Conversion

Plaintext converted to binary

02

Initial Permutation

IP applied to 64-bit block

03

Split Halves

Split into left and right 32-bit halves

04

16 Feistel Rounds

Iterative rounds with round keys

05

Final Permutation

IP^{-1} applied to produce ciphertext

Project Structure

The DES implementation is organized across six Python modules, each with a clear responsibility within the encryption pipeline.

File Tree

```
DES_Project/  
|  
├─ main.py  
├─ des_rounds.py  
├─ feistel.py  
├─ key_generation.py  
├─ utils.py  
└─ README.md
```

File Explanations

main.py

Controls the whole program — user interaction and CLI menu

des_rounds.py

Implements the 16 DES encryption rounds

feistel.py

Feistel function and all DES permutation tables

key_generation.py

Generates 16 round keys from the master key

utils.py

Helper functions for conversion and formatting

main.py — Program Controller

main.py is the entry point of the application. It orchestrates all other modules and provides the command-line interface for user interaction.

encrypt_message()

- Converts plaintext to binary
- Pads message to 64-bit blocks
- Encrypts block by block

decrypt_message()

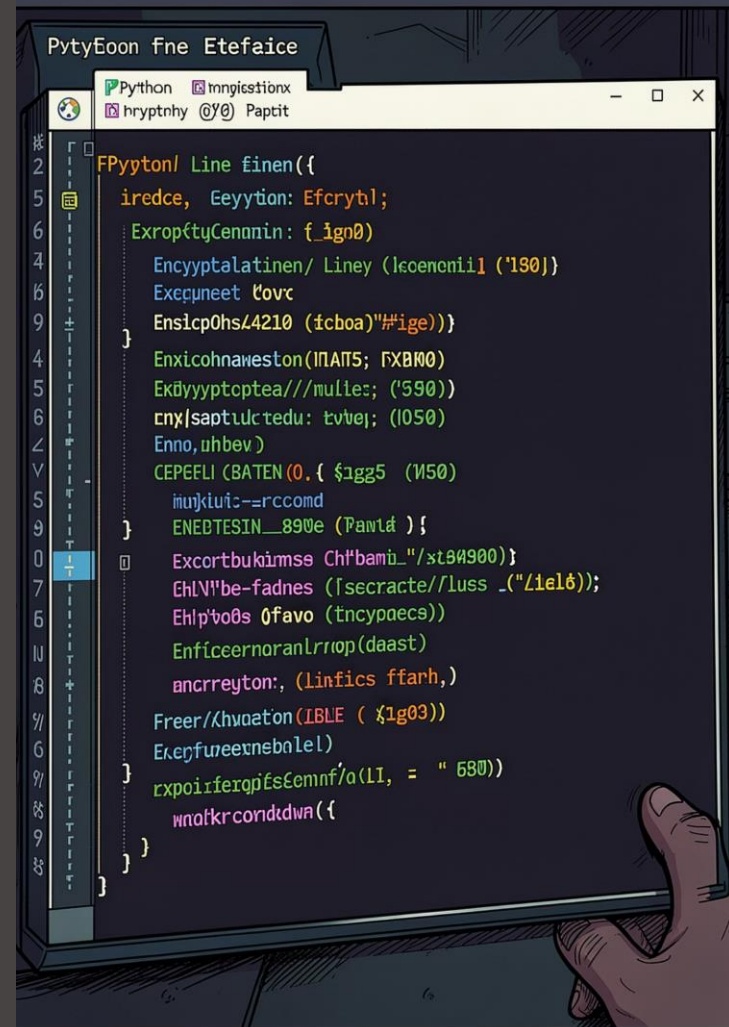
- Converts hex ciphertext to binary
- Uses reversed round keys
- Restores original plaintext

test()

- Runs predefined test cases
- Validates encryption output
- Verifies decryption accuracy

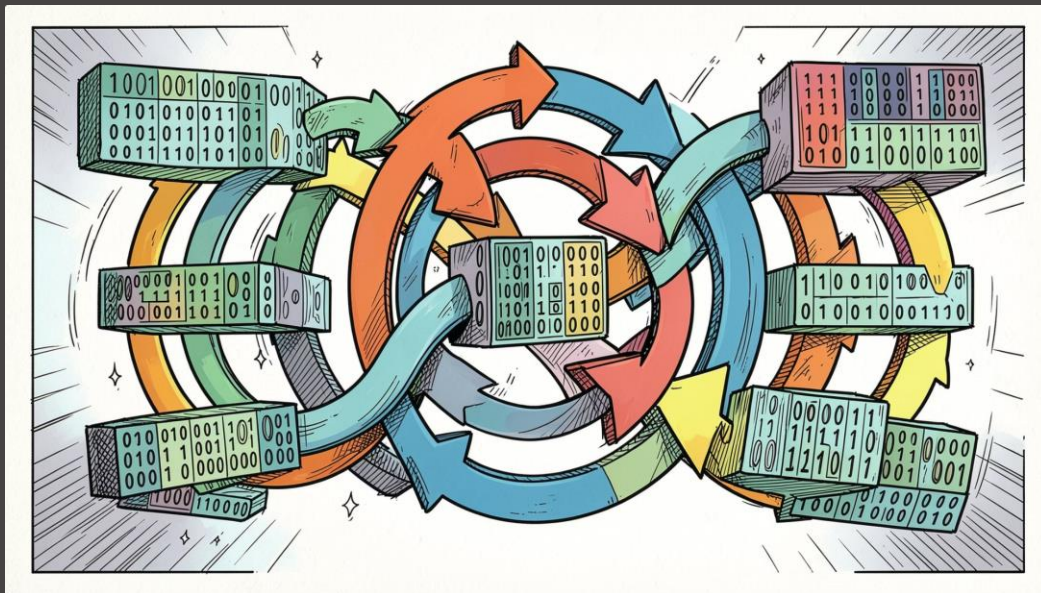
main()

- CLI menu interface
- Handles user input
- Calls appropriate functions



des_rounds.py — The 16 Feistel Rounds

This module implements the core iterative structure of DES — 16 rounds of Feistel processing that transform the plaintext block using round-specific subkeys.



Core Function: `rounds(l, r, keys)`

Performs all 16 Feistel rounds with left-right swapping and XOR operations.

Core DES Round Formula:

$$L_{i+1} = R_i, \quad R_{i+1} = L_i \oplus F(R_i, K_i)$$

1 Left becomes Right

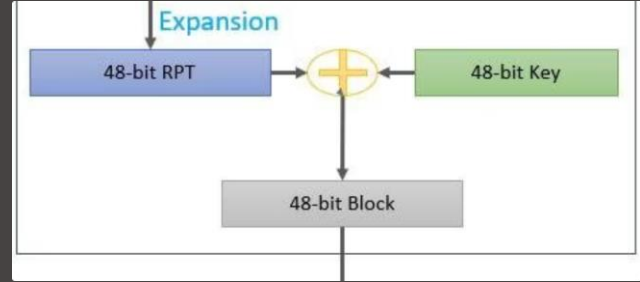
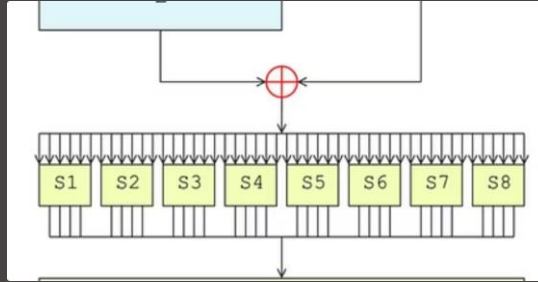
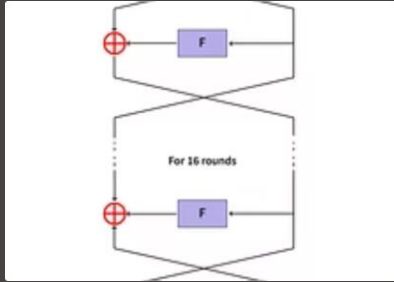
$$L_{i+1} = R_i$$

2 Right becomes XOR

Previous left XOR Feistel function output

feistel.py — Core Cryptographic Operations

This module contains all the fundamental DES operations including permutation tables, S-boxes, and the Feistel function implementation.



Initial Permutation (IP)

Rearranges input bits before processing begins.



S-Boxes

Convert 48 bits → 32 bits; introduce confusion.



Expansion Function

Expands 32 bits → 48 bits for key mixing.



P-Box

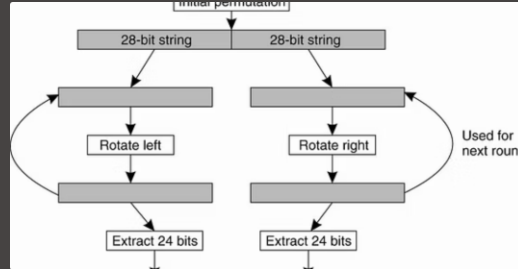
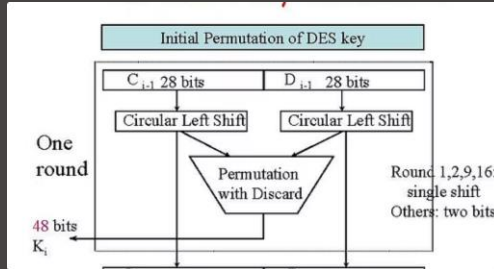
Permutes bits for diffusion across the block.



Feistel Function: $F(R, K) = P(S(E(R) \oplus K))$ — Expansion → XOR with key → S-box substitution → P-box permutation.

key_generation.py — DES Key Scheduling

This module generates all 16 round subkeys from the original 56-bit key through a series of permutations and left-shift operations.



Permuted Choice One (PC1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

7-Step Key Schedule

01

Apply PC-1

Permuted Choice 1 on 64-bit key

02

Split C & D

Divide into two 28-bit halves

03

Left Shift

Rotate bits each round

04

Apply PC-2

Permuted Choice 2 selection

05

Generate Keys

Produce all 16 round keys

③ **Key Schedule Formula:** $K_i = PC2(LS_i(C_o) \parallel LS_i(D_o))$

utils.py

Helper module providing utility functions for DES encryption and decryption.

Purpose

Provides helper functions for cryptographic operations.

Important Functions

- `text_to_binary()`
- `binary_to_text()`
- `binary_to_hex()`
- `hex_to_binary()`
- `permute()`
- `pad_plaintext()`
- `unpad_plaintext()`

What These Functions Simplify

Conversions

Text, binary, and hex transformations

Padding

Block alignment for DES

Permutations

Bit-level rearrangement

Encryption Process Example

Walkthrough of DES encryption using a simple example.

Example Input

Plaintext: HELLO123

Key: SECRET12

Step-by-Step Process

01

Convert to binary

02

Generate round keys

03

Initial permutation

04

16 rounds of Feistel

05

Final permutation

06

Ciphertext generated

Decryption Process

How Decryption Works

DES decryption uses the **same structure and algorithm** as encryption — the only difference is that round keys are applied in **reverse order**.

Key Insight

Because DES is a Feistel cipher, encryption and decryption are symmetric operations. Only the key schedule order changes.

 This symmetry is one of the elegant properties of Feistel network design.

Same DES structure

Same algorithm

Reverse order of keys

Mathematical Formulation

$$DES^{-1}(C) = DES(C, K_{16}, K_{15}, \dots, K_1)$$

Testing and Results

Test Cases

Hello123

Hi

Test

Single character

Empty string

Verification Method

The program verifies correctness by checking:

✓ **Plaintext == Decrypted Text**

Outcome

All tests passed successfully, confirming the implementation correctly encrypts and decrypts various input types.



Provides hands-on learning of core cryptography concepts through a working implementation.

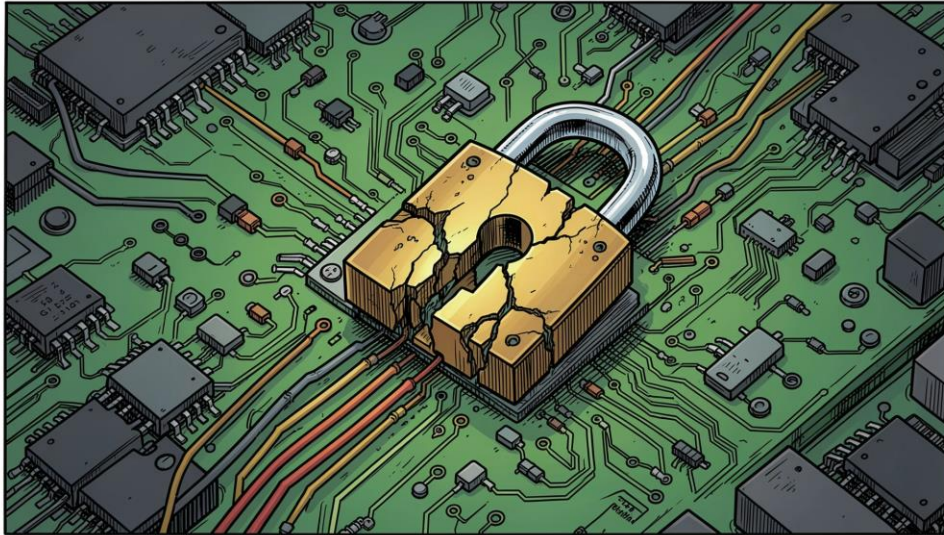


No external libraries — entirely written in Python for transparency and portability.



Clean, readable code that students and developers can study and extend.

Limitations



Known Weaknesses

DES is Outdated

Originally designed in the 1970s, DES no longer meets modern security standards.

56-Bit Key Vulnerable

A 56-bit key can be brute-forced with modern computing power in a practical timeframe.

ECB Mode is Insecure

Electronic Codebook mode reveals patterns in the plaintext, making it unsuitable for real data.

Simple Null-Byte Padding

The padding scheme lacks robustness and does not handle all edge cases securely.

Future Improvements



Optimize Performance

Improve speed and efficiency of the encryption and decryption routines.



Improve Padding Scheme

Replace null-byte padding with a more robust standard like PKCS#7.



Add GUI Interface

Build a graphical interface to make the tool accessible to non-technical users.



Add CBC Mode

Implement Cipher Block Chaining mode to eliminate ECB pattern vulnerabilities.



Implement AES

Extend the project to support the modern AES standard for stronger security.

Conclusion

What Was Achieved

Successfully implemented the full DES algorithm in pure Python, demonstrating both encryption and decryption processes end-to-end.

- Encryption and decryption process demonstrated
- All test cases passed
- Clean, educational implementation

Key Learnings

→ Encryption Techniques

Understanding how block ciphers transform plaintext into ciphertext.

→ Permutations

Bit-level rearrangement as a core cryptographic primitive.

→ Feistel Networks

The symmetric structure enabling reversible encryption.

→ Key Scheduling

How round keys are derived from the main key.

References

- 1 DES Standard
FIPS PUB 46-3 — Federal Information Processing Standards Publication, the official DES specification.
- 2 Cryptography Textbooks
Academic references covering symmetric encryption, Feistel structures, and block cipher modes.
- 3 Python Documentation
Official Python language reference for implementation details and standard library usage.
- 4 Educational DES Resources
Online tutorials and academic materials explaining DES internals for learners and developers.



Thank You